

ส่วนที่ 1: หลักการออกแบบ

1.1 อธิบายว่าใช้ SOLID ส่วนไหนบ้างและเกี่ยวข้องกับ Entity อย่างไร

S — Single Responsibility Principle

แยก Product, ProductDetail และ Review ออกเป็นคนละ class เพื่อรับผิดชอบข้อมูลเฉพาะส่วน โดย Product เก็บข้อมูลหลัก, ProductDetail เก็บข้อมูลเสริม เช่น description และ weight, ส่วน Review เก็บข้อมูลรีวิวนั่น ในระดับ Layer แยกหน้าที่ชัดเจนระหว่าง ProductController ที่รับส่ง HTTP request/response และ ProductService ที่จัดการเฉพาะ Business Logic เช่น การคำนวณส่วนลดและการผูก FK

O — Open/Closed Principle

ออกแบบโครงสร้างให้ขยายได้โดยไม่ต้องแก้ไขโค้ดเดิม การเพิ่ม Review ทำได้ผ่านการเชื่อม @OneToMany(mappedBy = "product") ที่ Product เพียงจุดเดียว สำหรับส่วนลดใช้ Strategy Pattern เมื่อต้องการเพิ่มโปรโมชั่นใหม่ เช่น ClearanceDiscountStrategy สามารถสร้าง class ใหม่ที่ implement DiscountStrategy แล้วเพิ่ม case ใน DiscountContext ได้ทันทีโดยไม่ต้องแก้ไข Strategy เดิม

L — Liskov Substitution Principle

Repository ทั้งหมด (ProductRepository, ProductDetailRepository, ReviewRepository) extends JpaRepository และสามารถใช้งานผ่าน Interface เดียวกันได้ทุกจุด เช่นเดียวกับ class ส่วนลดต่าง ๆ (NoDiscount, MemberDiscount, SeasonalSale) ที่สามารถใช้แทนกันผ่าน reference type DiscountStrategy โดยพฤติกรรมของ applyDiscount() ยังคงถูกต้องตาม contract เดิม

I — Interface Segregation Principle

แยก Repository ออกเป็น 3 interface ตาม Entity แทนการรวมไว้ใน interface เดียว เพื่อให้แต่ละส่วนเรียกใช้เฉพาะเมธอดที่เกี่ยวข้อง เช่นเดียวกับ DiscountStrategy ที่กำหนดเฉพาะเมธอด applyDiscount(double price) โดยไม่รวมเมธอดอื่นที่ไม่จำเป็น

D — Dependency Inversion Principle

ProductService ขึ้นกับ ProductRepository ซึ่งเป็น interface (abstraction) ไม่ใช่ implementation class ที่ Hibernate สร้างขึ้นจริง และรับผ่าน Constructor Injection

```
public ProductService(ProductRepository productRepository) {
    this.productRepository = productRepository;
    this.discountContext = new DiscountContext();
}
```

เช่นเดียวกัน ProductController ก็ขึ้นกับ ProductService ผ่าน constructor เดียวกัน ทำให้ high-level module (Service, Controller) ไม่ต้องรู้จัก low-level implementation detail

1.2 อธิบายความแตกต่างของ 1:1 กับ 1:N — ใช้กรณีไหนควรใช้แบบไหน

One-to-One (Product ↔ ProductDetail): เหมาะสำหรับข้อมูลที่มีได้เพียง 1 ชุดต่อ 1 รายการหลัก โดยใช้วิธีแยกตารางเพื่อลดขนาดของตารางหลักและหลีกเลี่ยงการดึงข้อมูลที่ไม่จำเป็น ตามหลัก SRP โดยให้ Product เป็นฝั่งที่ถือ FK

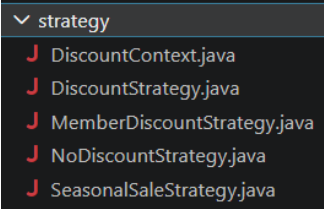
One-to-Many (Product → Review): เหมาะสำหรับข้อมูลส่วนลูกที่มีจำนวนไม่แน่นอนและขยายตัวได้เรื่อยๆ โดยกำหนดให้ฝั่ง Review (Many) เป็นผู้ถือ FK เพื่อรองรับโครงสร้างตารางเชิงสัมพันธ์
การเลือกใช้ หากโครงสร้างข้อมูลสัมพันธ์กันแบบ 1 ต่อ 1 แน่นนอน และต้องการความเป็นระเบียบในการจัดเก็บ ให้เลือกใช้ 1:1 แต่หากข้อมูลฝั่งลูกมีจำนวนไม่จำกัด ให้เลือกใช้ 1:N

1.3 อธิบาย Strategy Pattern สำหรับคำนวณส่วนลด

การใช้เงื่อนไขตรวจสอบ (if-else) จัดการส่วนลดหลายรูปแบบ จะส่งผลให้ต้องแก้ไขโค้ดเดิมทุกครั้งเมื่อมีเงื่อนไขใหม่ ซึ่งขัดต่อหลัก OCP ระบบจึงประยุกต์ใช้ Strategy Pattern โดยกำหนด DiscountStrategy เป็น Interface กลาง และแยก Logic การคำนวณส่วนลดแต่ละประเภทออกเป็น Class Implementation

```
package com.example.lab8_6733800371_sec1.strategy;

public interface DiscountStrategy {
    double applyDiscount(double price);
}
```



ระบบใช้ DiscountContext เป็นตัวกลางในการเลือก Strategy ตามประเภทส่วนลด ทำให้ ProductService สามารถเรียกใช้งานประมวลผลได้โดยตรง ส่งผลให้การเพิ่มโปรโมชั่นใหม่ทำได้โดยการสร้าง Class ใหม่ โดยไม่ส่งผลกระทบต่อชุดโค้ดเดิม

```
public class DiscountContext {

    public double calculate(String discountType, double price) {
        DiscountStrategy strategy = switch (discountType == null ? "NONE" : discountType) {
            case "MEMBER" -> new MemberDiscountStrategy();
            case "SEASONAL" -> new SeasonalSaleStrategy();
            default -> new NoDiscountStrategy();
        };
        return strategy.applyDiscount(price);
    }
}
```

1.4 อธิบาย Execution Flow ตั้งแต่ HTTP Request → DB

1. ผู้ใช้กด "บันทึกการแก้ไข" ที่หน้า edit.html → browser ส่ง POST /products/update/{id} พร้อมข้อมูลฟอร์ม

2. ProductController.update() รับ request, Spring MVC bind ข้อมูลฟอร์มเป็น object Product ผ่าน @ModelAttribute
3. Controller เรียก productService.getProductById(id) เพื่อดึงข้อมูลเดิม ใช้ preserve reviews และ id ของ detail เดิม กันปัญหา Hibernate สร้างแถวใหม่ซ้ำ
4. Controller เรียก productService.updateProduct(id, product)
5. ใน ProductService, เมธอด linkChildEntities() ผูก FK: set detail.setProduct(product) และ loop set review.setProduct(product) ให้ทุกรีวิว เพราะ JPA ไม่ผูก FK ให้อัตโนมัติ
6. productRepository.save(product) ถูกเรียก — Spring Data JPA แปลงเป็น SQL UPDATE/INSERT ตาม cascade ที่กำหนดไว้ (CascadeType.ALL) แล้วส่งไปที่ PostgreSQL ผ่าน Hibernate + JDBC driver
7. Database อัปเดตทั้ง 3 ตาราง (products, product_details, reviews) ให้สอดคล้องกัน
8. Controller ตั้งค่า flash message แล้ว redirect:/products
9. Browser ยิง GET /products ใหม่ → ProductController.list() เรียก productService.getAllProducts() → ดึงข้อมูลจาก DB มาแสดงพร้อมส่วนลดที่คำนวณใหม่ผ่าน DiscountContext

ส่วนที่ 2: Code + คำอธิบาย

2.1 Entity

Product

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String category;
    private String brand;
    private Integer stock;
    private Double price;
    private String discountType;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "detail_id", referencedColumnName = "id")
    private ProductDetail detail;

    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Review> reviews = new ArrayList<>();

    @Transient
    private Double discountedPrice;

    public Product() {}

    // getters/setters
```

- `@Entity, @Table(name="products")` บอก JPA ให้ map class นี้กับตาราง products ใน PostgreSQL
- `@Id @GeneratedValue(strategy = GenerationType.IDENTITY)` กำหนด primary key และให้ database เป็นผู้ generate ค่า
- `@OneToOne(cascade = CascadeType.ALL) @JoinColumn(name = "detail_id", ...)` ประกาศความสัมพันธ์ 1:1 โดย Product เป็นฝั่งเจ้าของ ถือคอลัมน์ FK ชื่อ detail_id ไว้ และ cascade = ALL หมายความว่าเมื่อ save/update/delete Product การกระทำเดียวกันจะกระทำกับ ProductDetail ที่ผูกอยู่ด้วยอัตโนมัติ
- `@OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)` ประกาศความสัมพันธ์ 1:N โดย Product เป็นฝั่ง inverse ไม่ถือ FK เอง mappedBy = "product" บอกว่า field product ใน Review เป็นผู้ถือ FK จริง ส่วน orphanRemoval = true ทำให้ถ้าลบ review ออกจาก list นี้ แถวใน DB จะถูกลบตามไปด้วย
- `@Transient` บอก JPA ว่าฟิลด์ discountedPrice ไม่ต้อง map ลงคอลัมน์ใดๆ ในฐานข้อมูล เพราะเป็นค่าที่คำนวณสดๆ ผ่าน Strategy Pattern ตอนดึงข้อมูลมาแสดงเท่านั้น

ProductDetail

```
@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(length = 1000)
    private String description;

    private String warranty;

    private Double weight;

    private String dimensions;

    private String manufacturedCountry;

    @OneToOne(mappedBy = "detail")
    private Product product;

    public ProductDetail() {}

    // getters/setters
}
```

- @Column(length = 1000) กำหนดความยาวคอลัมน์ description ให้รองรับข้อความยาวกว่าค่า default (255 ตัวอักษร)
- @OneToOne(mappedBy = "detail") เป็นฝั่ง inverse ของความสัมพันธ์ 1:1 ไม่ถือ FK เอง mappedBy ชี้กลับไปอยู่ที่ field detail ใน Product ทำให้ทราบว่า ProductDetail ตัวนี้เป็นของ Product ตัวไหน โดยไม่ต้องมีคอลัมน์ FK ซ้ำซ้อนในตาราง product_details

Review

```
@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String reviewer;

    private Integer rating;

    @Column(length = 1000)
    private String comment;

    private LocalDate reviewDate;

    @ManyToOne
    @JoinColumn(name = "product_id")
    private Product product;

    public Review() {}

    // getters/setters
}
```

- @ManyToOne @JoinColumn(name = "product_id") ประกาศว่า Review หลายแถวสามารถอ้างอิงถึง Product เดียวกันได้ และเป็นฝั่งที่ถือ FK จริง คือคอลัมน์ product_id ในตาราง reviews ตามกฎที่ว่า FK อยู่ฝั่ง Many เสมอ ในความสัมพันธ์ 1:N

2.2 Service

```
@Service
public class ProductService {

    private final ProductRepository productRepository;
    private final DiscountContext discountContext;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
        this.discountContext = new DiscountContext();
    }

    public List<Product> getAllProducts() {
        List<Product> products = productRepository.findAll();
        products.forEach(this::applyDiscount);
        return products;
    }
}
```

ProductService ประกาศ dependency (ProductRepository) เป็น private final และรับค่าผ่าน constructor เท่านั้น ทำให้ Spring จะ inject ProductRepository implementation ที่สร้างขึ้นอัตโนมัติเข้ามาตอนสร้าง bean ข้อดีของวิธีนี้คือ

- บังคับว่า object ต้องมี dependency ครบก่อนถูกสร้างขึ้น
- ทำให้ field เป็น final ได้ ป้องกันการเปลี่ยนแปลง dependency ภายหลังโดยไม่ตั้งใจ
- เขียน unit test ได้ง่าย เพราะสามารถ new ProductService(mockRepository) เพื่อ inject mock เข้าไปตรงๆ ได้โดยไม่ต้องพึ่ง Spring context
- สอดคล้องกับหลัก DIP เพราะ ProductService ขึ้นกับ interface ProductRepository ไม่ใช่ class ที่ Hibernate generate จริง

2.3 Controller

```
@Controller
@RequestMapping("/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public String list(Model model) {
        model.addAttribute("products", productService.getAllProducts());
        return "products/list";
    }

    @GetMapping("/add")
    public String addForm(Model model) {
        // ...
    }
}
```

เช่นเดียวกับ Service, ProductController รับ ProductService ผ่าน constructor เพียงตัวเดียว ทำให้ controller ทำหน้าที่แค่ "แปลง HTTP request เป็นเรียก service method แล้วส่ง view กลับ" (ตาม SRP) โดยไม่มี business logic เช่น การคำนวณส่วนลด หรือการผูก FK ปะปนอยู่ใน controller เลย logic เหล่านั้นถูกซ่อนอยู่หลัง ProductService ทั้งหมด ทำให้ถ้าวันหนึ่งต้องเปลี่ยนวิธีคำนวณส่วนลดหรือวิธี save ข้อมูล ก็แก้ที่ ProductService อย่างเดียว ไม่กระทบ ProductController

ภาพหน้าจอ

<http://localhost:8080/products>



เพิ่มสินค้า

<http://localhost:8080/products/add>

The screenshot shows the 'Add New Product' form. At the top, there's a header with a plus icon and the title '+ เพิ่มสินค้าใหม่'. Below the header is a section titled 'ข้อมูลสินค้า (Product)'. The form contains several input fields: 'ชื่อสินค้า' (Product Name) with the value 'iPhone 15 Pro (673380037-1 SEC 1)', 'หมวดหมู่' (Category) with the value 'Smartphone', 'ยี่ห้อ' (Brand) with the value 'Apple', 'จำนวนในคลัง' (Inventory Count) with the value '80', 'ราคาปกติ (บาท)' (Regular Price (Baht)) with the value '39900', and 'ประเภทส่วนลด' (Discount Type) with a dropdown menu showing 'ไม่มีส่วนลด' (No discount).

📄 ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

สมาร์ตโฟนระดับพรีเมียมจาก Apple

ระยะเวลาประกัน

1 Year

น้ำหนัก (กิโลกรัม)

0.18

ขนาด (กว้าง x ยาว x สูง)

146.6x70.6x8.25 mm

ประเทศที่ผลิต

China

★ รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N

เพิ่มรีวิวพร้อมกับสินค้า (สามารถเพิ่มรีวิวเพิ่มเติมภายหลังได้)

ชื่อผู้รีวิว

Hello World

คะแนน (1-5)

★★★★☆ (4)

ความเห็น

สินค้าคุณภาพดี

📅 บันทึกสินค้า

ยกเลิก



Product Shop

Lab 8 — 1:1 & 1:N Relationship

เพิ่มสินค้าเรียบร้อยแล้ว

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
1	iPad air 6 (673380037-1 SEC 1)	Electronics	Apple	100	25000.00	NONE	25000.00	2 Year	0.5	1 รีวิว	แก้ไขลบ
3	iPhone 15 Pro (673380037-1 SEC 1)	Smartphone	Apple	80	39900.00	NONE	39900.00	1 Year	0.18	1 รีวิว	แก้ไขลบ

ตาราง Products

Query

Query History

↗

Scratch Pad ✕

1

SELECT * FROM public.products

2

ORDER BY id ASC

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🔍

📥

📶

Showing rows: 1 to 2

Page No: 1 of 1

⏪

⏩

	id [PK] bigint	brand character varying (255)	category character varying (255)	discount_type character varying (255)	name character varying (255)	price double precision	stock integer	detail_id bigint
1	1	Apple	Electronics	NONE	iPad air 6 (673380037-1 SEC 1)	25000	100	1
2	3	Apple	Smartphone	NONE	iPhone 15 Pro (673380037-1 SEC...	39900	80	4

ตาราง Product_details

```
1 SELECT * FROM public.product_details
2 ORDER BY id ASC
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1

	id [PK] bigint	description character varying (1000)	dimensions character varying (255)	manufactured_country character varying (255)	warranty character varying (255)	weight double precision
1	1	ชิป Apple M2	178.5x247.6x0.5 mm	USA	2 Year	0.5
2	4	สมาร์ตโฟนระดับพรีเมียมจาก Ap...	146.6x70.6x8.25 mm	China	1 Year	0.18

แก้ไขสินค้า

http://localhost:8080/products/edit/{id}

localhost:8080/products/edit/3

แก้ไขสินค้า

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iPhone 15 Pro (673380037-1 SEC 1)

ยี่ห้อ

Apple

ราคาปกติ (บาท)

39900.0

หมวดหมู่

Smartphone

จำนวนในคลัง

68

ประเภทส่วนลด

ส่วนลดเทศกาล (20%)

ลบสินค้า

http://localhost:8080/products/products/delete/{id}

localhost:8080/products/delete/3

☆

 ยืนยันการลบสินค้า

คุณต้องการลบสินค้านี้ใช่ไหม? การดำเนินการนี้ไม่สามารถย้อนกลับได้

ชื่อสินค้า: iPhone 15 Pro (673380037-1 SEC 1)

หมวดหมู่: Smartphone

ยี่ห้อ: Apple

ราคา: 39900.00 บาท

รายละเอียด: สมาร์ทโฟนระดับพรีเมียมจาก Apple

รับประกัน: 2 Year

⚠ ข้อมูลเสริม (ProductDetail) จะถูกลบพร้อมกันด้วย เนื่องจาก CascadeType.ALL

ยืนยันลบ

ยกเลิก

 Product Shop

Lab 8 — 1:1 & 1:N Relationship

ลบสินค้าเรียบร้อยแล้ว

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
1	iPad air 6 (673380037-1 SEC 1)	Electronics	Apple	100	25000.00	NONE	25000.00	2 Year	0.5	1 รีวิว	แก้ไข ลบ

Database 3 ตาราง

ตาราง Products

```
1 SELECT * FROM public.products
2 ORDER BY id ASC
```

	id [PK] bigint	brand character varying (255)	category character varying (255)	discount_type character varying (255)	name character varying (255)	price double precision	stock integer	detail_id bigint
1	1	Apple	Electronics	NONE	iPad air 6 (673380037-1 SEC...	25000	100	1

ตาราง Product_details

```
1 SELECT * FROM public.product_details
2 ORDER BY id ASC
```

	id [PK] bigint	description character varying (1000)	dimensions character varying (255)	manufactured_country character varying (255)	warranty character varying (255)	weight double precision
1	1	ชิป Apple M2	178.5x247.6x0.5 mm	USA	2 Year	0.5

ตาราง reviews

```
1 SELECT * FROM public.reviews
2 ORDER BY id ASC
```

	id [PK] bigint	comment character varying (1000)	rating integer	review_date date	reviewer character varying (255)	product_id bigint
1	1	คุ้มครับ	5	2026-09-03	นายเจ	1